

3. Design

DEEP LOCAL



Student No.	22112128
Name	변도영
E-mail	dybyun2002@naver.com

[Revision history]

Revision date	Version #	Description	Author
2026/3/27	1.00	초기 버전	변도영
2026/4/30	1.01	기능 구현	
2026/6/5	1.02	기능 추가	

= Contents =

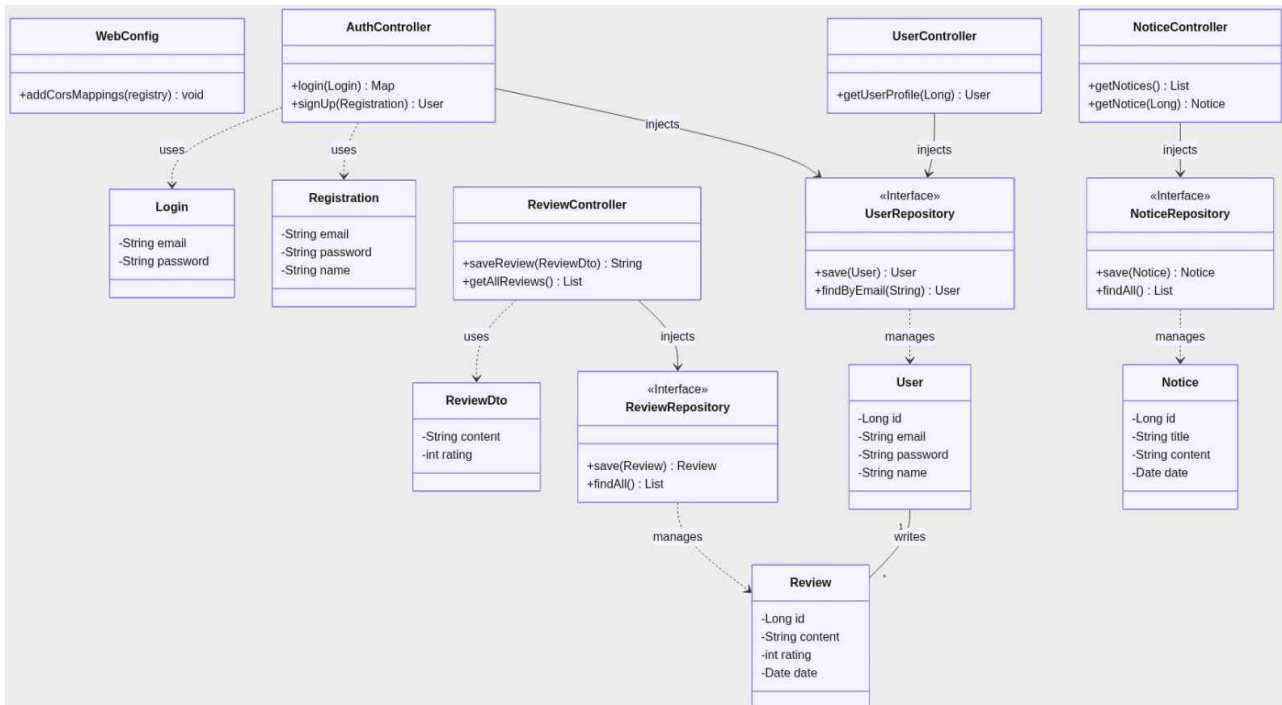
1. Introduction	
2. Class diagram	
3. Sequence diagram	
4. State machine diagram	
5. Implementation requirements	
6. Glossary	
7. References	

1. Introduction

전 세계에 산재한 잘 알려지지 않은 관광 명소를 발굴하고, 사용자 간의 진솔한 리뷰 공유를 통해 정보의 불균형을 해소하기 위한 'DEEP LOCAL' 시스템의 설계 및 분석 내용을 기술한다. 본 시스템은 기존의 유명 관광지 중심의 획일화된 여행에서 벗어나, 방문 국가의 진정한 분위기와 고유한 문화를 깊이 있게 체험하고자 하는 현대 여행자들의 니즈를 충족시키는 데 그 목적이 있다. 특히 정보의 '희소성'이 곧 가치가 되는 로직을 도입하여, 아직 대중화되지 않은 장소들을 데이터화하고 이를 여행자들과 연결하는 플랫폼으로서의 역할을 수행한다. 시스템은 리뷰 수에 따른 동적 해시태그 관리와 포인트 보상 체계를 핵심 동력으로 삼으며, 이를 객체지향 설계 기법을 통해 안정적인 시스템으로 구현하고자 한다.

본 프로젝트가 가진 주요 특징과 차별점은 다음과 같다. 단순히 텍스트 정보를 제공하는 것에 그치지 않고, Google Maps API와 연동하여 숨겨진 장소의 정확한 좌표 및 상세 주소를 실시간으로 제공함으로써 여행자의 편의성을 극대화한다. 또한 글로벌 사용자를 고려한 다국어 설정 기능을 통해 국가 간 정보 격차를 줄이고 폭넓은 탐색 환경을 지원한다. 그리고 정보의 가치가 변질되지 않도록 '희소성'에 대한 정량적 기준 (리뷰 100개 미만)을 수립하였다. 기준 초과 시 #새로운, #잘알려지지않은과 같은 특정 태그를 시스템이 자동 소멸시킴으로써 정보의 신선도를 유지하고, 사용자가 직접 '숨은 명소'를 발견하고 선점하는 차별화된 경험을 제공한다. 또한, 사용자가 신뢰도 높은 리뷰를 게시할 경우 활동 보상인 포인트를 지급하며, 이를 외부 은행 API와 실시간 환율 정보를 활용하여 실제 해당 국가의 화폐로 환전할 수 있는 금융 연동 기능을 갖춘다. 이는 양질의 숨은 정보를 자발적으로 공유하게 만드는 강력한 기제로 작용한다. 마지막으로, 팔로우 및 댓글 기능을 포함한 SNS 기반의 구조로 설계되어 사용자 간의 영향력을 형성하고 커뮤니티를 구축할 수 있다. 이러한 설계는 향후 현지 가이드 매칭 서비스, 지역 기반 문화 큐레이션 등 다양한 비즈니스 모델로의 확장이 용이하다는 강점을 가진다.

2. Class diagram



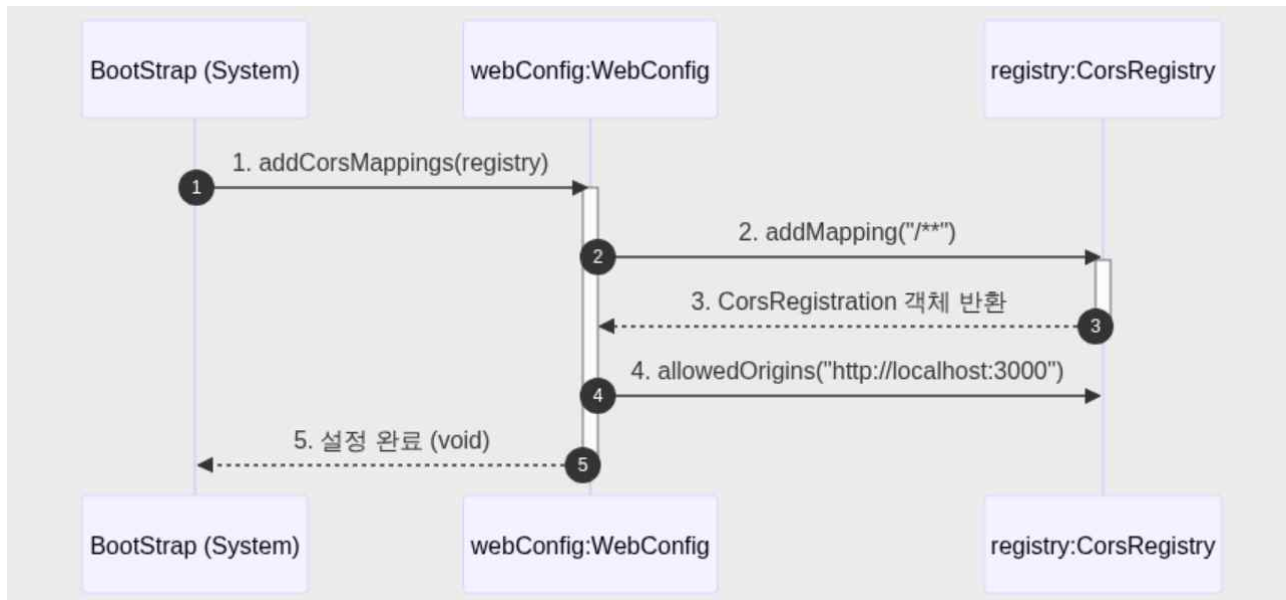
class	Explanation
User	시스템에 가입한 회원의 고유 정보를 안전하게 보관하기 위해 모두 private(-)으로 숨겨두었다. 데이터가 DB에 들어가기 전 기본값은 null 또는 빈 문자열("")로 설정된다. 식별자인 id, 로그인에 쓰이는 email, 암호화되어 저장될 password, 그리고 실명이나 닉네임을 담는 name이 있다. 이 속성들은 반드시 1개([1]) 존재해야 한다. 숨겨진 속성값을 안전하게 밖으로 꺼내볼 수 있도록 getId, getEmail, getName 같은 읽기 전용 기능(public(+))을 제공한다. 또한, 사용자가 자신의 이름을 변경할 수 있도록 updateProfile(name) 연산을 제공하여 객체 스스로 자신의 상태를 변경하게 한다.
Review	사용자가 남긴 평가 기록이다. 리뷰의 고유 식별자 id, 평가 내용인 content, 별점인 rating, 그리고 리뷰가 작성된 시간을 기록하는 createdAt으로 구성된다.

	작성 시간은 객체가 생성될 때 현재 시간(new Date())으로 자동 초기화된다. 리뷰 정보를 읽어오기 위한 getId, getContent, getRating 연산이 있다. 만약 리뷰를 수정해야 한다면, 내용과 별점이 동시에 바뀌는 것이 논리적으로 맞으므로 updateContent(content, rating)이라는 하나의 연산으로 두 가지 속성을 동시에 업데이트한다.
Notice	관리자가 작성한 공지글 정보이다. 식별자 id, 글 제목 title, 본문 내용 content, 작성일 createdAt으로 구성된다. 일반 사용자는 공지를 읽기만 하므로 getId, getTitle, getContent와 같이 데이터를 조회하는 연산만 제공한다.
Login	로그인 요청 시 이메일과 비밀번호는 항상 세트로 움직인다. 따라서 이 두 속성(email, password)을 묶어 하나의 객체로 만들었다. 입력된 이메일과 비밀번호를 검증 로직에 전달하기 위해 getEmail(), getPassword() 연산을 제공한다.
Registration	회원가입 시에는 더 많은 정보가 필요하다. email, password에 추가로 name 속성까지 묶어서 서버로 한 번에 전달한다. 가입 처리를 위해 각각의 값을 꺼낼 수 있는 읽기 연산(getEmail, getPassword, getName)을 제공한다.
ReviewDto	새로운 리뷰를 등록할 때 프론트엔드에서 서버로 보내는 데이터 꾸러미이다. 작성한 본문(content), 별점(rating), 그리고 누가 작성했는지 알기 위한 회원 번호(userId)를 속성으로 가진다. 저장 로직에 이 값들을 전달할 수 있도록 각각의 읽기 연산을 제공한다.
UserRepository	회원 정보를 DB에 저장하는 save(user), 회원 번호로 유저를 찾는 findById(id), 이메일로 유저를 찾는 findByEmail(email) 연산이 있다. 이때 반환값의 다중성을 [0..1]로 설정한 이유는, 해당 이메일이나 아이디를 가진 유저가 DB에 없을 수도 있기 때문이다.
ReviewRepository	작성된 리뷰를 DB에 저장하는 save(review)와, DB에 있는 모든 리뷰 데이터를 목록 형태로 가져오는 findAll() 연산을 제공한다.
NoticeRepository	관리자가 쓴 공지를 저장하는 save(notice), 전체 공지 목록을

	가져오는 findAll(), 특정 공지글 하나를 찾아오는 findById(id) 연산이 있다.
WebConfig	시스템 전반의 네트워크 규칙을 설정하는 역할을 한다. addCorsMappings(registry) 연산을 통해 React 화면(보통 3000번 포트)과 Spring Boot 서버(8080번 포트) 간에 데이터가 막힘없이 오갈 수 있도록 통신 허용 경로(CORS)를 등록한다.
AuthController	가입 및 로그인 처리를 위해 DB에 접근해야 하므로 userRepository를 private 속성으로 가진다. - login(loginData): 닉네임의 아이디/비밀번호 대신 앞서 만든 Login 복합 객체를 통째로 넘겨받아 로그인을 처리하고 그 결과를 반환한다. - signUp(regData): Registration 객체를 넘겨받아 회원가입 로직을 수행하고 생성된 User 객체를 반환한다.
UserController	유저 정보를 조회/수정하기 위해 userRepository를 속성으로 가진다. 특정 회원의 아이디(id)를 넘겨받아 프로필을 보여주는 getUserProfile과, 수정된 정보를 받아 프로필을 갱신하는 updateProfile 연산을 제공한다.
ReviewController	리뷰 데이터와 작성자 데이터를 모두 다뤄야 하므로 reviewRepository와 userRepository 두 가지를 속성으로 가진다. - saveReview(reviewDto): 프론트엔드에서 넘어온 ReviewDto 바구니를 받아 DB에 저장하고 성공 메시지를 반환한다. - getAllReviews(): 화면에 장소의 모든 리뷰를 띄워주기 위해 리뷰 목록(Review[*])을 반환합니다. 다중성 [*]은 여러 개의 데이터가 리스트 형태로 나감을 의미한다.
NoticeController	공지사항 데이터 접근을 위해 noticeRepository를 가진다. 전체 공지사항 목록을 불러오는 getNotices()와, 특정 공지글 하나만 클릭해서 볼 때 사용하는 getNotice(id) 연산을 제공한다.

3. Sequence diagram

1. Web 전역에 대한 흐름



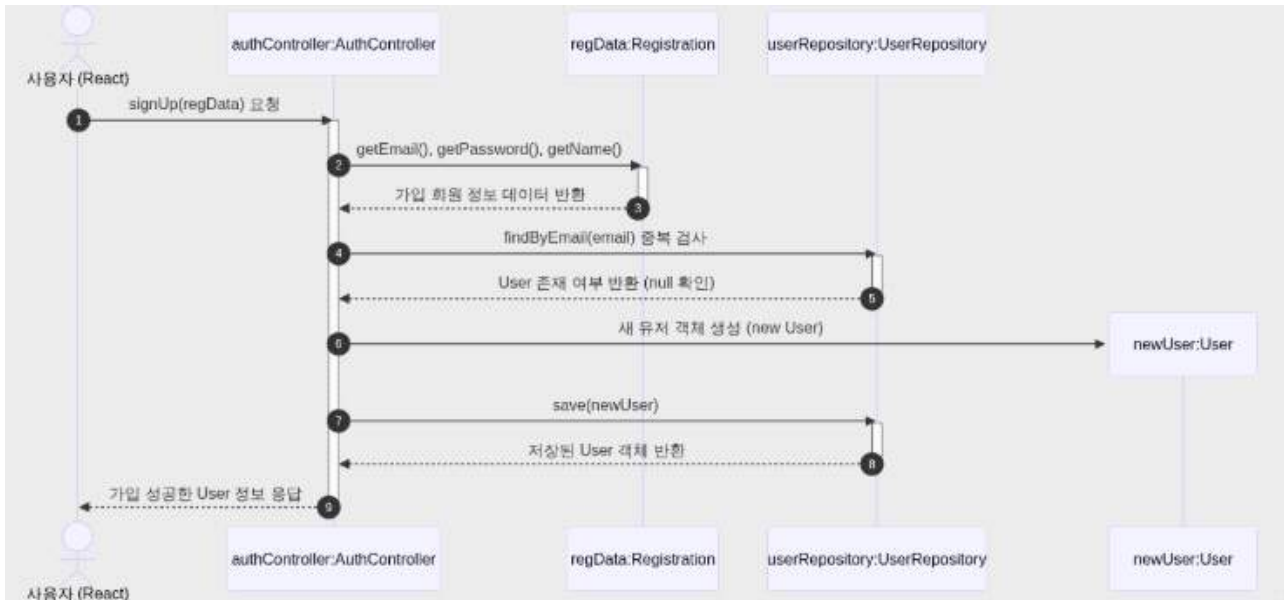
시스템이 최초 구동(BootStrap)될 때, 클라이언트(React)와 서버(Spring Boot) 간의 원활한 통신을 가로막는 브라우저의 교차 출처 리소스 공유(CORS) 보안 정책을 조율하는 초기화 상호작용이다.

시스템 엔진이 구동되면서 전역 설정을 담당하는 WebConfig 클래스의 addCorsMappings 오퍼레이션을 호출하며 설정 도구인 CorsRegistry 객체를 인자로 주입한다.

WebConfig 객체는 주입받은 registry 인스턴스에 접근하여 모든 API 경로(/)에 대한 통신을 허용하는 매핑 규칙을 선언한다.

이어서 프론트엔드가 동작하는 고유 출처 주소(http://localhost:3000)를 허용 호스트(allowedOrigins)로 등록함으로써 전역 보안 설정을 안전하게 마친 후 제어권을 시스템에 반환한다.

2. 회원가입



프론트엔드로부터 전달받은 가입 정보를 검증하고, 도메인 규칙에 따라 엔티티 객체를 동적으로 생성하여 저장소(DB)에 영속화하는 데이터 흐름이다.

클라이언트(React)가 가입 정보가 담긴 복합 데이터 타입 구조체인 Registration(regData) DTO를 인자로 하여 AuthController의 signUp 오퍼레이션을 호출한다.

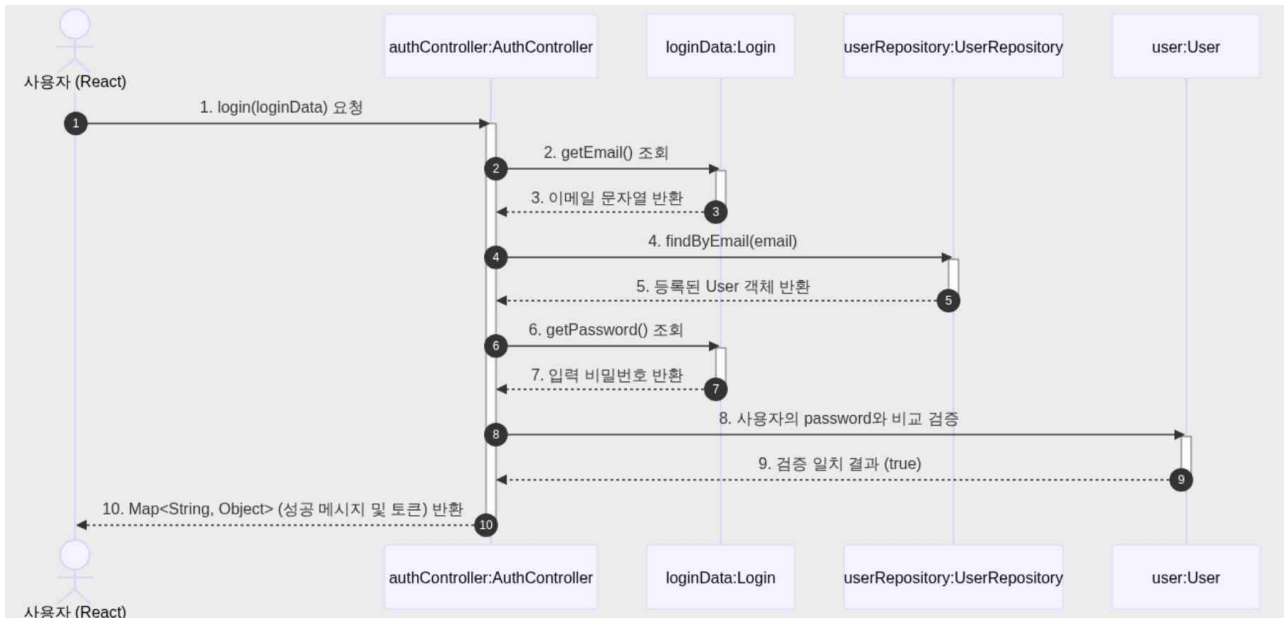
요청을 받은 컨트롤러는 인자로 넘어온 regData 객체의 Getter 연산(getEmail, getPassword, getName)을 통해 캡슐화된 사용자 입력값을 안전하게 추출한다.

추출된 데이터 중 중복 가입을 방지하기 위해 UserRepository 인터페이스의 findByEmail 연산을 호출하여 데이터베이스 내 기존 회원 존재 여부를 조회한다.

중복이 없음이 확인되면(결과물 null), AuthController는 User 엔티티 객체를 메모리상에 동적으로 생성(new User)한다.

최종적으로 변경된 유저 객체를 UserRepository.save 연산에 전달하여 영속성 컨텍스트(DB)에 반영하고, 성공적으로 데이터베이스에 안착한 User 객체를 클라이언트에 응답한다.

3. 로그인



클라이언트가 전송한 인증 복합 데이터를 바탕으로 등록된 회원의 정보와 비밀번호의 유효성을 검증하고 그 결과를 반환하는 보안 제어 흐름이다.

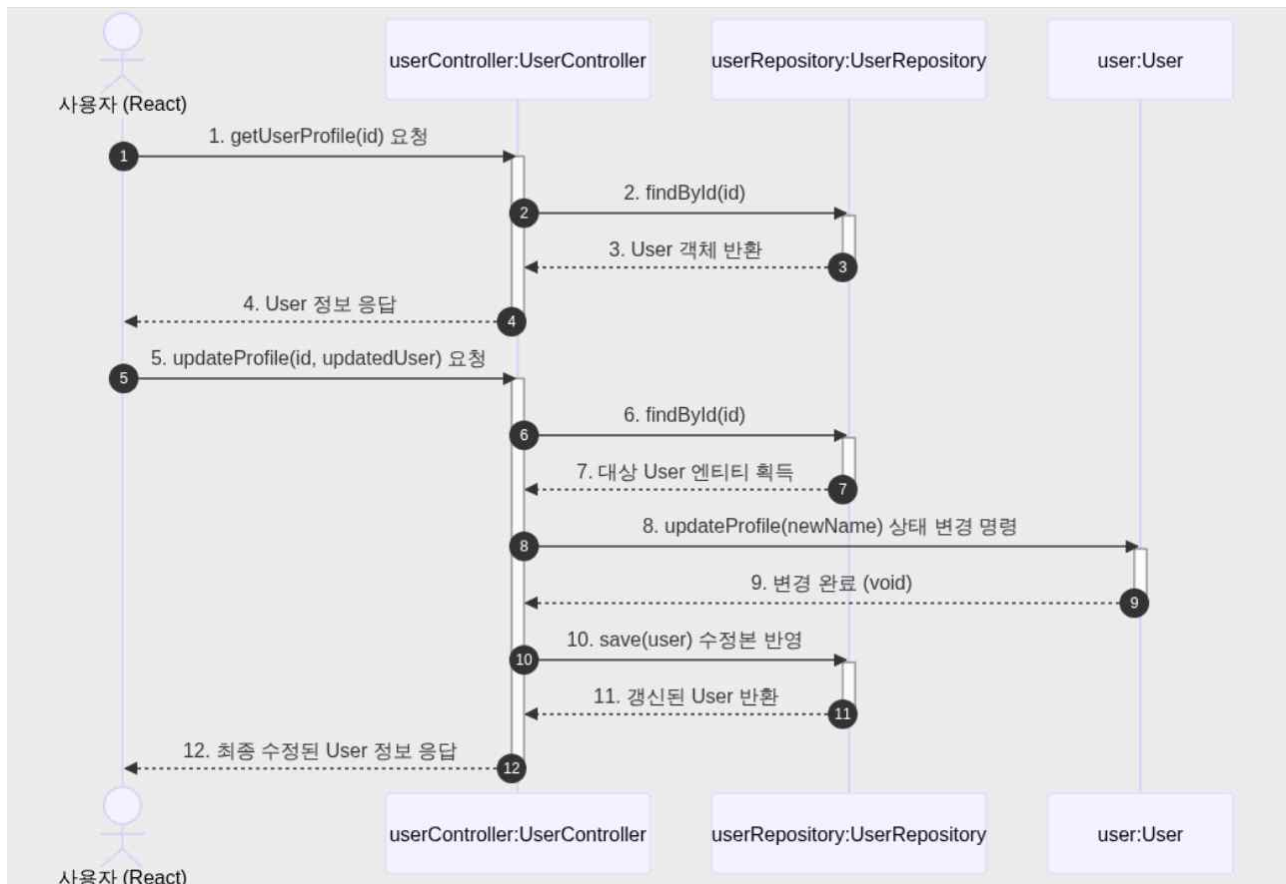
사용자가 아이디와 패스워드를 입력하면, 이 변수들이 묶인 Login(loginData) 복합 객체가 AuthController.login 연산의 매개변수로 전달된다.

AuthController는 loginData.getEmail() 연산으로 식별자를 뽑아낸 후, UserRepository.findByEmail을 실행해 영속화되어 있던 실제 User 엔티티를 찾아온다. 유저 엔티티가 정상적으로 반환되면, 컨트롤러는 다시 DTO에서 사용자가 방금 입력한 비밀번호(getPassword)를 추출한다.

데이터 보안을 위해 컨트롤러가 직접 비밀번호를 비교하지 않고, User 엔티티 객체의 내부 연산에 입력 비밀번호를 넘겨 객체 스스로가 자격 증명을 검증하도록 유도한다.

검증 결과가 일치(true)로 확인되면 시스템은 로그인 성공 메시지와 보안 토큰 등을 구조화된 Map<String, Object> 형태로 묶어 클라이언트에 반환한다.

4. 마이페이지 회원 정보 관리



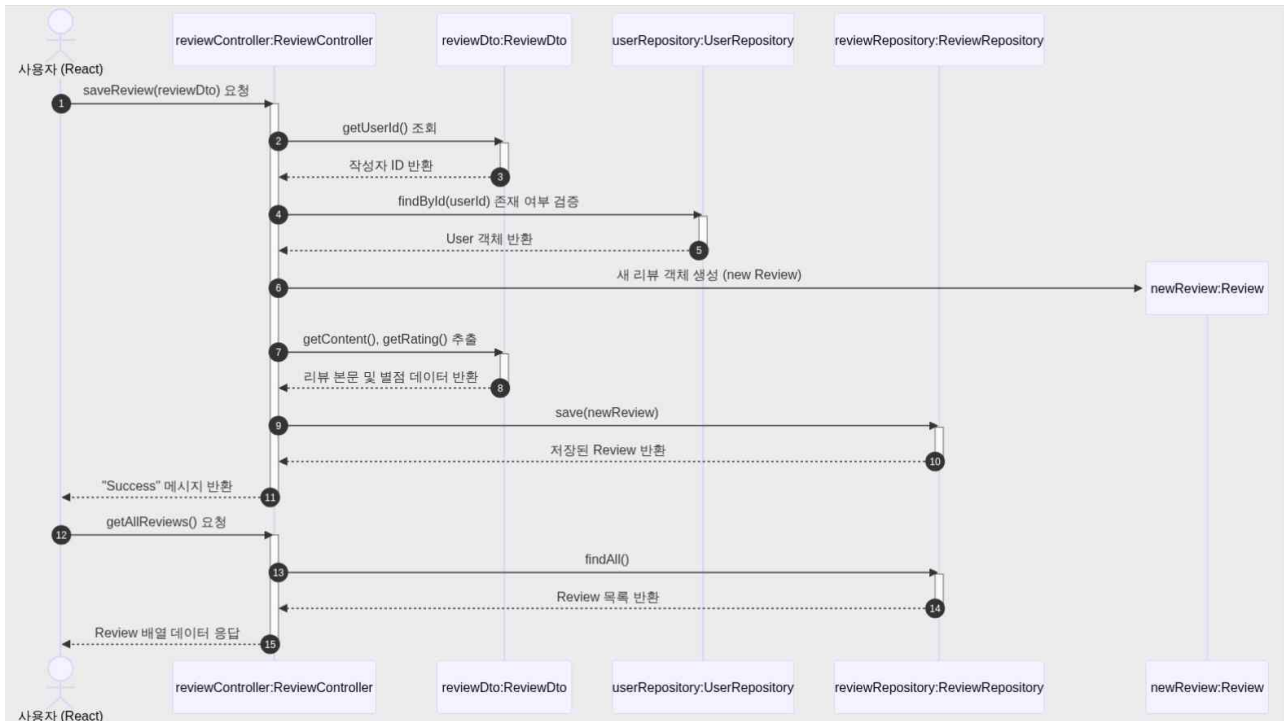
마이페이지 진입 시 회원의 고유 독립 정보를 화면에 랜더링하고, 정보 변경 발생 시 엔티티의 공통 연산을 수행하여 프로필 상태를 갱신하는 흐름이다.

프로필 조회 흐름: 클라이언트가 회원 고유 번호(id)를 지참하여 UserController.getUserProfile을 호출하면, 컨트롤러는 UserRepository.findById를 통해 매핑되는 User 엔티티 정보를 통째로 획득하여 화면에 뿌려준다.

프로필 수정 흐름: 사용자가 이름을 바꾸고 저장 버튼을 누르면 updateProfile(id, updatedUser) 오퍼레이션이 발동한다.

컨트롤러는 먼저 수정 대상이 될 원래의 User 엔티티를 DB에서 꺼내온 뒤, 해당 유저 엔티티의 updateProfile(newName) 연산을 직접 명하여 객체의 상태 속성을 변경한다. 상태가 바뀐 유저 엔티티는 다시 UserRepository.save에 의해 DB에 마킹되고 최종 갱신된 정보가 리턴된다.

5. 리뷰 작성 및 열람

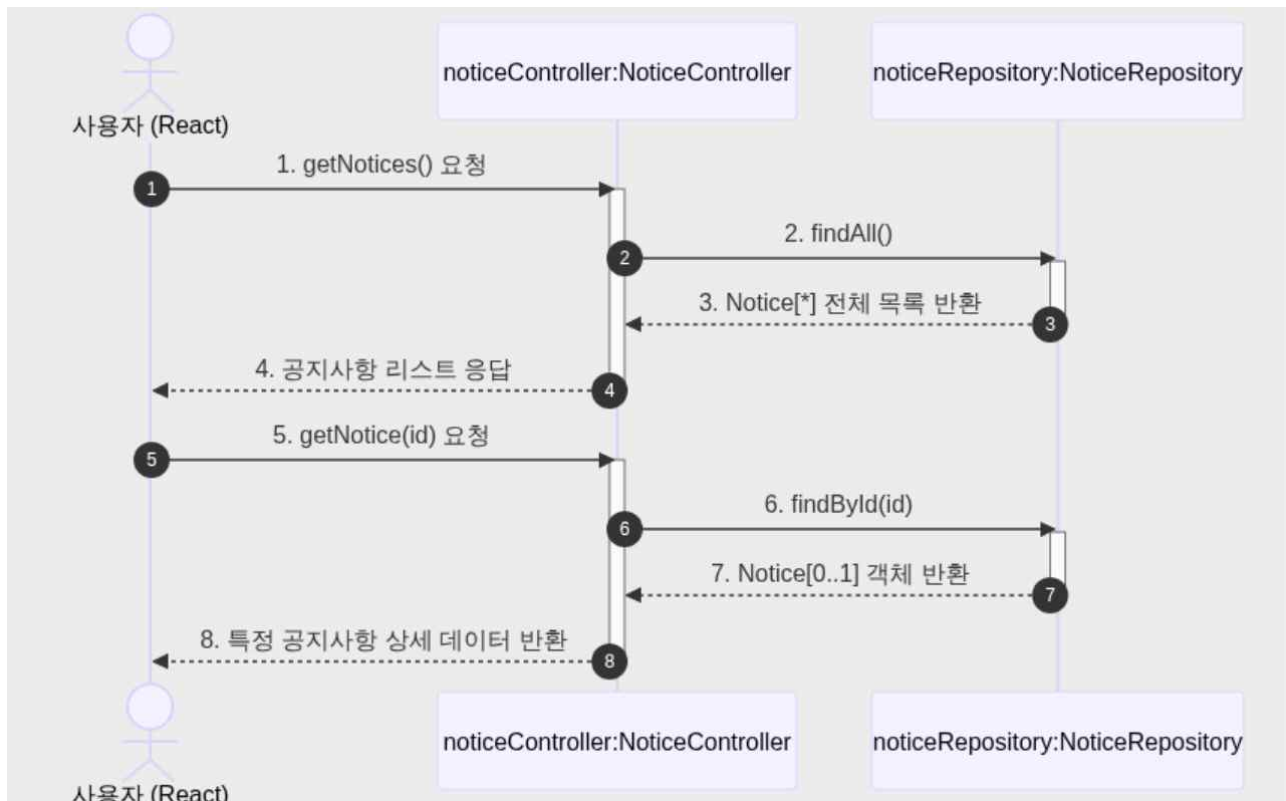


특정 장소나 서비스에 대해 작성된 리뷰 데이터를 수집·검증·저장하고, 화면 요청 시 데이터베이스에 누적된 리뷰 목록을 다중성([*])에 맞춰 일괄 전송하는 상호작용이다.

리뷰 저장 흐름: 사용자가 글과 별점을 남기면 본문, 평점, 작성자 ID가 응집된 ReviewDto 바구니가 ReviewController.saveReview로 유입되고, 컨트롤러는 먼저 Dto.getUserId()로 얻은 아이디가 실제 유효한 회원인지 UserRepository에서 검증한다. 검증이 완료되면 데이터의 주체인 Review 엔티티를 동적으로 생성하고 DTO 속성값들을 바인딩한 후, ReviewRepository.save를 통해 영속화시킨 뒤 성공 문자열("Success")을 응답한다.

리뷰 목록 조회 흐름: 장소 상세 페이지에 접속하여 ReviewController.getAllReviews를 호출하면, 제어가 ReviewRepository.findAll 오퍼레이션을 트리거한다. 저장소는 현재 누적된 리뷰들을 UML 다중성 규칙인 Review[*] 배열(리스트) 형태로 반환하며, 컨트롤러는 이 컬렉션 데이터를 클라이언트에 그대로 전송한다.

6. 공지사항 조회

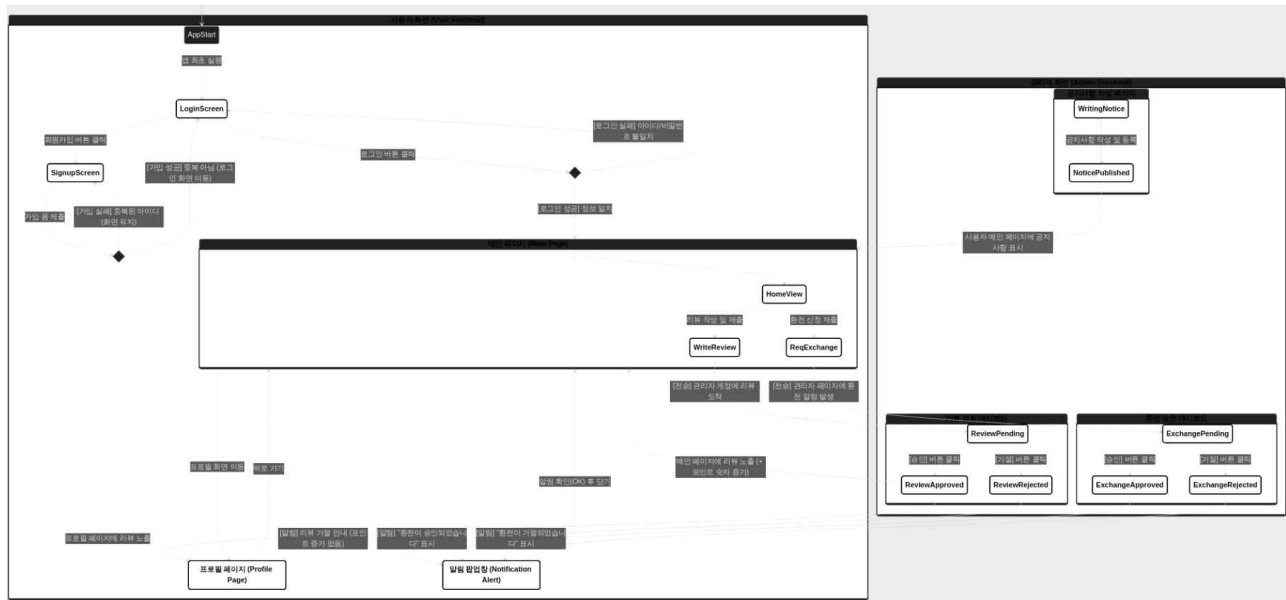


전체 공지사항 리스트를 호출하는 다중 조회 유즈케이스와 특정 공지사항의 상세 본문을 호출하는 단일 조회 유즈케이스를 처리하는 제어 흐름이다.

전체 목록 조회 흐름: 사용자가 공지사항 탭을 누르면 NoticeController.getNotices() 메서드가 호출된다. 컨트롤러는 즉시 NoticeRepository.findAll() 메서드를 호출하여 DB 내부의 모든 공지 엔티티들을 다중성 리스트(Notice[*]) 형태로 읽어모아 화면에 배열 데이터로 전달한다.

상세조회흐름: 목록 중 특정 공지글 하나를 클릭하면 글의 일련번호와 함께 NoticeController.getNotice(id) 요청이 서버로 들어온다. 영속성 인터페이스인 NoticeRepository.findById()가 호출되어 데이터가 존재하지 않을 수도 있는 예외 상황을 고려한 다중성 Notice[0..1] 형태로 객체를 안전하게 뽑아내어 사용자에게 상세 본문 내용을 제공한다.

4. State machine diagram



State	Explanation
LoginScreen	사용자가 아이디와 비밀번호를 입력하는 인증 대기 화면. 가입되지 않은 사용자는 회원가입 화면으로 전환할 수 있는 버튼이 제공된다.
SignupScreen	신규 사용자가 이용할 아이디와 비밀번호 등 회원 정보를 입력하는 화면 상태이다.
SignupCheck	입력한 회원가입 정보를 검증하는 화면 내부 로직 상태이다. 중복 아이디 존재 시: 실패 팝업을 띄우고 기존 입력 내용이 유지된 채 SignupScreen으로 돌아간다. 중복 아이디 없음: 가입 완료 메시지와 함께 자동으로 LoginScreen으로 화면이 전환된다.
LoginCheck	사용자가 입력한 아이디와 비밀번호의 유효성을 프론트엔드에서 일차적으로 판별하는 상태이다. 정보 불일치: "아이디 또는 비밀번호가 틀렸습니다"라는 안내와 함께 LoginScreen에 머무른다. 정보 일치: 인증 토큰을 브라우저에 저장하며 사용자의 핵심 공간인 MainPage로 진입한다.
MainPage	로그인에 성공한 인증 유저들이 머무르는 상위 컨테이너 화면. 다음 3가지 하위 뷰(View) 상태를 포함한다. HomeView (홈 화면): 메인 페이지의 기본 화면으로 다른

	유저들의 승인된 리뷰와 관리자가 올린 공지사항이 렌더링 되어 표시되는 공간. 리뷰 작성 폼이나 환전 신청 단추가 위치한다. WriteReview (리뷰 작성 상태): 사용자가 리뷰 폼을 열고 텍스트와 별점을 입력하는 상태. [전송]을 누르면 화면의 데이터가 관리자 대시보드로 넘어간다. ReqExchange (환전 신청 상태): 사용자가 원하는 환전 금액을 입력하는 폼 상태. 제출 완료 시 관리자 화면에 즉시 환전 요청 팝업 알림이 트리거된다.
ProfilePage	사용자의 개인 요약 정보(마이페이지)를 보여주는 화면입니다. 이곳에는 관리자로부터 최종 승인을 받은 리뷰 목록만 누적되어 표시된다.
UserAlert	관리자가 행한 승인/거절 행동에 대한 피드백을 사용자 화면 최상단에 띄워주는 모달 팝업 상태. 사용자가 확인 버튼을 누르면 알림창이 닫히며 MainPage로 돌아온다.
AdminReview	관리자가 사용자들이 제출한 리뷰들을 모니터링하고 제어하는 화면 영역이다. ReviewPending : 사용자가 작성한 리뷰가 관리자 대시보드 리스트에 등록되어 검토를 기다리는 상태. ReviewApproved : 관리자가 리뷰 내용에 문제가 없다고 판단하여 [승인] 버튼을 클릭한 상태. 이 이벤트가 발생하면 사용자 화면의 MainPage와 ProfilePage에 해당 리뷰가 실시간 렌더링되며, 유저의 포인트 UI 숫자가 즉시 증가한다. ReviewRejected : 관리자가 부적절한 리뷰로 판단하여 [거절] 버튼을 클릭한 상태. 이 경우 유저의 포인트는 증가하지 않으며, 사용자 화면에 거절 안내 팝업을 트리거한다.
AdminExchange	사용자들이 신청한 포인트 환전 내역을 관리하고 승인하는 화면 영역이다. ExchangePending : 사용자가 환전을 신청하여 관리자 페이지 우측 상단이나 리스트에 환전 요청 알림이 생성되어 대기하는 상태. ExchangeApproved : 관리자가 실제 송금을 완료한 후 [승인]을 클릭한 상태. 사용자 화면에 "환전이 승인되었습니다"라는 UserAlert를 보낸다. ExchangeRejected : 환전 조건 미달 등의 사유로 관리자

	가 [거절]을 클릭한 상태. 사용자 화면에 "환전이 거절되었습니다"라는 UserAlert를 보낸다
AdminNotice	시스템 전역 공지사항을 생성하고 배포하는 관리자 전용 화면 영역.

5. Implementation requirements

구분	요구사항
Hardware Requirements	CPU : Intel Core i5 이상 RAM : 4GB 이상 SSD or HDD : 256 GB 이상(SDD 권장)
Software Requirements	OS : Windows 10/11, macOS 10.15 이상 JDK : java OpenJDK 17 이상 IDE : IntelliJ IDEA or Eclipse 빌드 도구 : Gradle v8.x framework : Spring Boot v3.x
Network Requirements	연결되어있어야 함(API 요청)
Database Requirements	DBMS : MySQL v8.0 이상

6. Glossary

TERMS	Description
Architecture	시스템을 구성하는 소프트웨어 크기와 구조, 그리고 이들이 서로 상호작용하는 방식을 정의한 전체적인 설계도
DTO	계층 간(예: 프론트엔드 화면에서 백엔드 컨트롤러로) 데이터를 주고받을 때 오직 데이터만 담아서 나르는 바구니 역할
State	웹 페이지 화면에서 동적으로 변하는 데이터
Component	웹 화면을 구성하는 독립적이고 재사용 가능한 최소 UI 단위
Modal / Alert	기존 화면 위에 겹쳐서 나타나는 새로운 창
Repository	소프트웨어 코드와 실제 데이터베이스(DB) 사이를 연결하는 창구
Entity	데이터베이스의 테이블과 1:1로 매핑되는 백엔드 소스 코드 내부의 도메인 객체
JPA	자바 진영에서 사용하는 ORM 기술의 표준 인터페이스
JWT	인증에 필요한 정보들을 비밀키로 암호화하여 담은 가벼운 토큰 문자열
BCrypt	암호화를 거치면 다시는 원래의 평문 문장으로 복호화할 수 없는 보안 기술
WebSocket	클라이언트와 서버가 연결을 끊지 않고 양방향으로 데이터를 주고받는 실시간 통신 프로토콜

7. References

Spring Boot 공식 프로젝트 가이드: <https://spring.io/projects/spring-boot>

Spring Data JPA 공식 문서: <https://spring.io/projects/spring-data-jpa>

Mermaid.js 공식 가이드 및 라이브 편집기: <https://mermaid.js.org>

MySQL 공식 레퍼런스 매뉴얼: <https://dev.mysql.com/doc>